

SIMATS ENGINEERING

ASSESSMENT TOOL 1 - Experiment

COURSE NAME: Data Structures COURSE CODE: CSA0310

Priya dharshini M(192521051)

COURSE OUTCOME COVERED

CO: Solve problems for optimized data storage and retrieval using Binary Search Trees, AVL Trees, B-Trees, Red-Black Trees, and Splay Trees (BL3, BL5)

Assessment Tool Weightage: 40%

QUESTION 17

Write a program to construct a Splay Tree and implement zig, zig-zig, and zig-zag rotations. Then display each step of transformation.

Aim: To construct a Splay Tree and visualize every rotation step during insertion.

Algorithm / Implementation Steps

1. Create a Splay Tree node with key, left pointer and right pointer.
2. Implement left rotation and right rotation.
3. Implement Zig rotation when the accessed node is a direct child of the root.
4. Implement Zig-Zig rotation for two rotations in the same direction.
5. Implement Zig-Zag rotation for two rotations in opposite directions.
6. After every individual rotation, display the current tree.
7. Repeat until the accessed or inserted node becomes the root.

C PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int key;
    struct Node *left;
    struct Node *right;
};

struct Node* createNode(int key) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->key = key;
    n->left = NULL;
    n->right = NULL;
    return n;
}

struct Node* rightRotate(struct Node* y) {
    struct Node* x = y->left;
    struct Node* t = x->right;
    x->right = y;
    y->left = t;
    return x;
}

struct Node* leftRotate(struct Node* x) {
    struct Node* y = x->right;
    struct Node* t = y->left;
    y->left = x;
    x->right = t;
    return y;
}

void display(struct Node* root, int space) {
    if (root == NULL) return;

    space += 6;
    display(root->right, space);

    printf("\n");
    for (int i = 6; i < space; i++)
        printf(" ");
    printf("%d\n", root->key);

    display(root->left, space);
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}

void pauseStep(void) {
    printf("\nPress ENTER for next step...");
    getchar();
}

/* Splay operation with rotation-by-rotation display */
struct Node* splay(struct Node* root, int key) {
```

```

if (root == NULL || root->key == key)
    return root;

if (key < root->key) {
    if (root->left == NULL)
        return root;

    /* Zig-Zig */
    if (key < root->left->key) {
        root->left->left =
            splay(root->left->left, key);

        root = rightRotate(root);
        printf("\nZig-Zig: First Right Rotation\n");
        display(root, 0);
        pauseStep();
    }
    /* Zig-Zag */
    else if (key > root->left->key) {
        root->left->right =
            splay(root->left->right, key);

        if (root->left->right != NULL) {
            root->left = leftRotate(root->left);
            printf("\nZig-Zag: First Left Rotation\n");
            display(root, 0);
            pauseStep();
        }
    }
}

if (root->left == NULL)
    return root;

root = rightRotate(root);
printf("\nZig: Final Right Rotation\n");
display(root, 0);
pauseStep();

return root;
}

/* key > root->key */
if (root->right == NULL)
    return root;

/* Zig-Zig */
if (key > root->right->key) {
    root->right->right =
        splay(root->right->right, key);

    root = leftRotate(root);
    printf("\nZig-Zig: First Left Rotation\n");
    display(root, 0);
    pauseStep();
}
/* Zig-Zag */
else if (key < root->right->key) {
    root->right->left =
        splay(root->right->left, key);

    if (root->right->left != NULL) {
        root->right = rightRotate(root->right);
        printf("\nZig-Zag: First Right Rotation\n");
        display(root, 0);
        pauseStep();
    }
}

```

```

    }
}

if (root->right == NULL)
    return root;

root = leftRotate(root);
printf("\nZig: Final Left Rotation\n");
display(root, 0);
pauseStep();

return root;
}

struct Node* insert(struct Node* root, int key) {
    if (root == NULL)
        return createNode(key);

    root = splay(root, key);

    if (root->key == key)
        return root;

    struct Node* n = createNode(key);

    if (key < root->key) {
        n->right = root;
        n->left = root->left;
        root->left = NULL;
    } else {
        n->left = root;
        n->right = root->right;
        root->right = NULL;
    }

    return n;
}

void freeTree(struct Node* root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main(void) {
    struct Node* root = NULL;
    int n, key;

    printf("SPLAY TREE STEP-BY-STEP SIMULATION\n");
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    getchar();

    printf("Enter %d elements:\n", n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &key);
        getchar();

        printf("\n===== \n");
    }
}

```

```

printf("Inserting %d\n", key);
printf("=====\n");

root = insert(root, key);

printf("\nTree after insertion:\n");
display(root, 0);
printf("\nInorder: ");
inorder(root);
printf("\n");

if (i < n - 1)
    pauseStep();
}

printf("\nFinal Splay Tree:\n");
display(root, 0);

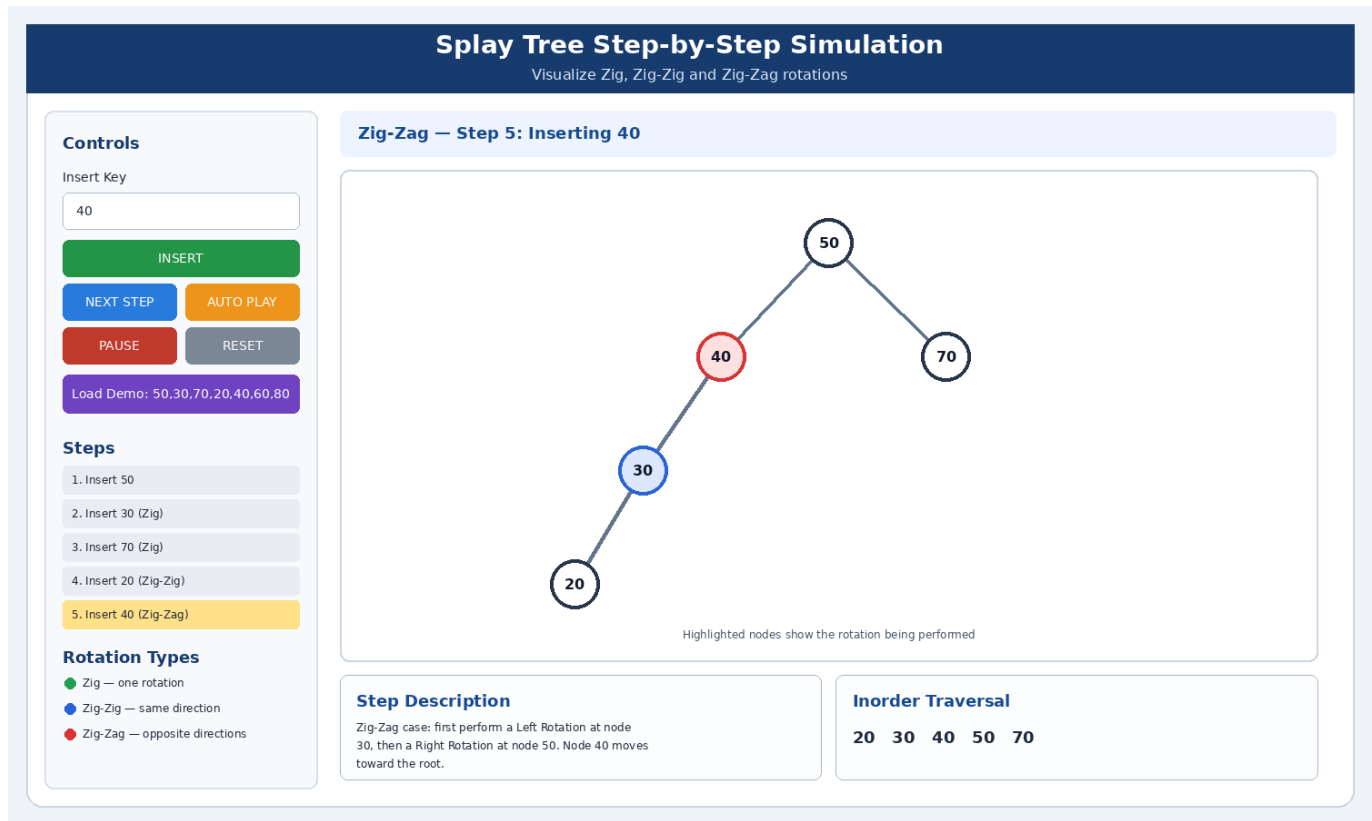
printf("\nFinal Inorder Traversal: ");
inorder(root);
printf("\n");

freeTree(root);
return 0;
}

```

IMPLEMENTATION / SIMULATION SCREENSHOT

The screenshot below shows the visual simulation with the complete tree, highlighted rotation nodes, step list, operation description, and inorder traversal.



Sample Input: 50 30 70 20 40 60 80

Expected Inorder Traversal: 20 30 40 50 60 70 80

Result: The Splay Tree was constructed successfully and Zig, Zig-Zig, and Zig-Zag transformations are displayed step-by-step.